

WhatsApp -> Salesforce

How I built it · what to say when asked

The one-paragraph answer

When a customer sends a WhatsApp message, Twilio receives it. Twilio is the messaging provider — it talks to WhatsApp for us, so we don't connect to Meta directly. Twilio then calls a webhook in Salesforce: a public URL that listens for incoming data. I built that URL as an Apex REST service (WhatsAppWebhookRestService), exposed through a Salesforce Site with a guest user. The service does three things: it checks a secret key so only Twilio can call it, it creates a Lead from the message (phone, email, and text), and it replies automatically to the customer. I also added two production-grade details: idempotency — if Twilio retries the same message, we still create only one Lead, because I track the unique MessageSid — and error logging, so no failure is lost silently.

If they go deeper

"Is this production-ready?"

The code is. For production we switch the Twilio Sandbox number to a verified WhatsApp Business number from Meta, validate Twilio's signature instead of a query secret, and add GDPR consent — important for the DACH market. Same Apex code.

"Why a Lead and not a Case?"

A Lead because it is a new, unknown contact reaching out. For an existing customer asking for support, the same pattern would create a Case instead — a small change in the service.

"Why Apex and not MuleSoft?"

For a single, simple inbound webhook, Apex is direct and fast. With many systems, or if we needed retries, queuing and routing, I'd put it behind MuleSoft — same contract, only the URL changes. That matches how I built the SAP layer.

Honesty rule (protects you)

Do NOT call this "Salesforce Headless Identity". It is a webhook / API integration (Twilio -> Apex REST -> Lead). If someone uses the wrong label, correct it gently — that precision makes you look senior, not junior.

Is this novel? (build vs buy)

No — WhatsApp-to-CRM is common, and Salesforce offers it natively through Digital Engagement (a paid Service Cloud add-on). I built the custom webhook version to show the engineering, not to claim a new feature. Custom = no extra license and full control of routing and object mapping (Lead today, Case for known customers). Native = faster if you already pay for Digital Engagement. Knowing both — and when to choose each — is the senior signal. In DACH I would gate it behind DSGVO consent, not capture every message.

WhatsApp -> Salesforce

Wie ich es gebaut habe · was du im Gespräch sagst

Die Antwort in einem Absatz

Wenn ein Kunde eine WhatsApp-Nachricht sendet, empfängt Twilio sie. Twilio ist der Messaging-Anbieter — er spricht für uns mit WhatsApp, also verbinden wir uns nicht direkt mit Meta. Dann ruft Twilio einen Webhook in Salesforce auf: eine öffentliche URL, die auf eingehende Daten wartet. Diese URL habe ich als Apex-REST-Service gebaut (WhatsAppWebhookRestService), über eine Salesforce-Site mit einem Gast-Benutzer. Der Service macht drei Dinge: Er prüft einen geheimen Schlüssel, damit nur Twilio ihn aufrufen kann, er erstellt einen Lead aus der Nachricht (Telefon, E-Mail und Text) und er antwortet automatisch dem Kunden. Ich habe auch zwei produktionsreife Details hinzugefügt: Idempotenz — wenn Twilio dieselbe Nachricht erneut sendet, erstellen wir trotzdem nur einen Lead, weil ich die eindeutige MessageSid speichere — und Fehler-Logging, damit kein Fehler still verloren geht.

Wenn sie tiefer fragen

"Ist das produktionsreif?"

Der Code ja. Für die Produktion wechseln wir die Twilio-Sandbox-Nummer zu einer verifizierten WhatsApp-Business-Nummer von Meta, prüfen die Twilio-Signatur statt eines Query-Secrets und fügen DSGVO-Einwilligung hinzu — wichtig für den DACH-Markt. Der Apex-Code bleibt gleich.

"Warum ein Lead und kein Case?"

Ein Lead, weil es ein neuer, unbekannter Kontakt ist. Für einen bestehenden Kunden mit einer Support-Anfrage würde dasselbe Muster einen Case erstellen — nur eine kleine Änderung im Service.

"Warum Apex und nicht MuleSoft?"

Für einen einzelnen, einfachen eingehenden Webhook ist Apex direkt und schnell. Bei vielen Systemen, oder wenn wir Retries, Queuing und Routing bräuchten, würde ich es hinter MuleSoft legen — gleicher Vertrag, nur die URL ändert sich. So habe ich auch die SAP-Schicht gebaut.

Ehrlichkeits-Regel (zu deinem Schutz)

Nenne das NICHT "Salesforce Headless Identity". Es ist eine Webhook-/API-Integration (Twilio -> Apex REST -> Lead). Wenn jemand das falsche Label benutzt, korrigiere es freundlich — genau diese Präzision lässt dich senior wirken, nicht junior.

Ist das neu? (build vs buy)

Nein — WhatsApp-zu-CRM ist verbreitet, und Salesforce bietet es nativ über Digital Engagement an (ein kostenpflichtiges Service-Cloud-Add-on). Ich habe die Custom-Webhook-Variante gebaut, um das Engineering zu zeigen, nicht um ein neues Feature zu behaupten. Custom = keine zusätzliche Lizenz und volle Kontrolle über Routing und Objekt-Mapping (heute Lead, Case für bekannte Kunden). Nativ = schneller, wenn man Digital Engagement schon bezahlt. Beides zu kennen — und zu wissen, wann man was wählt — ist das Senior-Signal.

Aussprache-Tipps

Idempotenz = "ee-dem-po-TENTS" · Gast-Benutzer = "gast be-NUT-ser" · eingehender Webhook = "AYN-gay-ender web-hook"